Software Architecture Document: Library Voice Agent

1. Executive Summary & System Overview

- **High-Level Purpose:** The Library Voice Agent is an interactive, real-time conversational AI system designed to act as a virtual library assistant ("Sam"). It allows users to query library catalogs, access research guides, and inquire about library policies using natural language voice commands, receiving accurate spoken responses in real-time.
- **Agent Type & Capabilities:** The system operates as a **Retrieval-Augmented Generation (RAG) conversational agent** integrated into a low-latency **Speech-to-Speech (S2S)** pipeline. It autonomously ingests multi-format documents (PDFs, DOCX, XLSX, CSV, TXT), maintains a semantic vector space, and executes conversational workflows triggered by voice input.

---

2. Technology Stack & Architectural Justifications

Component / Tool	Tech Used	Purpose	Architectural Justification (Why Chosen?)
Agent Framework	LangChain (`langchain-google-genai`)	Orchestration of RAG pipelines and prompt management.	Industry-standard modularity for document splitting, embedding integrations, and easy swap-out of LLMs.
Model Provider	Google Gemini (`gemini-flash-latest`)	Core reasoning, synthesis, and response generation.	Blazing fast inference speeds (vital for real-time speech interaction) and highly generous free-tier API limits.
Vector Store	ChromaDB	Persistent local semantic search and embedding storage.	Lightweight, serverless local database (SQLite-based) that avoids external cloud dependencies and network latency.
Embeddings	HuggingFace (`all-`	Converting raw	High-performance,

Component / Tool	Tech Used	Purpose	Architectural Justification (Why Chosen?)
	`MiniLM-L6-v2`)	document text into dense semantic vectors.	low-latency open-source model that runs completely locally without API costs.
Backend Runtime	Python 3.12 / FastAPI / Uvicorn	High-performance asynchronous API server.	FastAPI provides native async support, vital for handling simultaneous WebSocket audio streams and non-blocking LLM calls.
Frontend Runtime	HTML5 / Vanilla JS	Client-side interface for microphone capture and audio playback.	Zero-dependency, lightweight footprint ensuring broad browser compatibility via standard Web Speech APIs.

3. System Architecture & End-to-End Workflow Pipeline

Architectural Diagram

```
flowchart TD
    subgraph Client [Client-Side Frontend]
        UI[Browser UI]
        STT[Web Speech STT / Mic Capture]
        TTS[Web Speech TTS / Audio Playback]
    end

    subgraph Backend [FastAPI Server]
        API[main.py: API Endpoints]
        RAG[rag_engine.py: RAG Engine]
    end

    subgraph Database [Local Storage]
        Docs[(data/: Raw Files)]
        Chroma[(ChromaDB)]
    end
```

```
    subgraph External [External APIs]
        Gemini[Google Gemini API]
    end

    UI -->|Voice Input| STT
    STT -->|Transcribed Text| API
    API -->|Query| RAG
    Docs -->|Ingestion| RAG
    RAG <-->|Fetch/Store Embeddings| Chroma
    RAG <-->|Augmented Prompt| Gemini
    Gemini -->|Text Response| RAG
    RAG -->|Text Response| API
    API -->|Response| TTS
    TTS -->|Spoken Audio| UI
```

Step-by-Step Execution Lifecycle

1. **Input Ingestion & Transcription:** The user speaks into the client browser. The frontend utilizes standard Web Speech API (or custom WebRTC streaming) to capture and transcribe the audio into raw text.
2. **State Initialization & Routing:** The frontend POSTs the text to the FastAPI backend (`/api/chat`). The request is routed to the `LibraryRAG` instance.
3. **Retrieval (RAG):** The `LibraryRAG` engine embeds the incoming query using the local HuggingFace `MiniLM` model, then performs a similarity search against the ChromaDB vector store to fetch relevant chunks of library data.
4. **Agent Decision Loop (LLM Call):** The retrieved context and the user's question are injected into a strict system prompt ("You are Sam, a library assistant..."). This is sent to the Gemini API (`gemini-flash-latest`).
5. **Output Formatting & Response Delivery:** The LLM returns a comprehensive text response. The backend forwards this response to the frontend, where the Web Speech Synthesis API converts the text back into female-voiced audio.

---

4. File Structure & Dependency Mapping

Directory Tree Breakdown (Truncated to Core Systems)

```
C:\
|-- index.html                # Main Frontend Entry Point
|-- start.bat                 # Primary Windows initialization script
|-- run_local.bat             # Local runner with secured environment variab
les
|-- backend/
|   |-- main.py               # FastAPI ASGI application & routing
|   |-- rag_engine.py         # Core RAG orchestration and database logic
|   |-- requirements.txt      # Core backend dependencies
|   \-- chroma_db/            # Local SQLite persistent vector storage
|-- data/
```

```
|   |-- books_catalog.csv      # Structured library inventory
|   |-- library_policies.docx  # Unstructured library rules
|   \-- research_guide.pdf     # Unstructured PDF guides
\-- src/
    \-- speech_to_speech/      # Advanced modular Python S2S pipeline tools
        |-- LLM/               # LLM integration adapters
        |-- STT/               # Speech-to-Text handlers (Whisper, etc.)
        |-- TTS/               # Text-to-Speech handlers
        \-- VAD/               # Voice Activity Detection (Silero)
```

Import & Dependency Graph

- `backend/main.py` depends on `fastapi` and imports `backend/rag_engine.py` (specifically the `LibraryRAG` class).
- `backend/rag_engine.py` depends on `langchain`, `langchain-google-genai`, `langchain-community` (for `HuggingFaceEmbeddings`), and `chromadb`. It interacts heavily with the local filesystem (`data/` directory).
- `src/speech_to_speech/` serves as an extensible SDK for complex streaming (WebSockets, MLX, Whisper) that can be swapped in for local high-fidelity TTS/STT if browser APIs are insufficient.

---

5. Detailed Component Specifications

`backend/`

- **Purpose:** Central brain of the RAG application and HTTP server.
- **Key Classes:**
  - `LibraryRAG`: Handles document loading via `DirectoryLoader`, text splitting via `RecursiveCharacterTextSplitter`, embeddings, and the LangChain QA Chain.
- **Inputs:** Transcribed text queries from the user.
- **Outputs:** Plain-text LLM responses augmented with semantic context.

`data/`

- **Purpose:** The absolute source of truth for the RAG engine.
- **Dependencies:** `unstructured`, `pandas`, `pypdf`, `openpyxl`, and `docx2txt` are required by LangChain to parse the wide variety of MIME types present in this directory.

`src/speech_to_speech/`

- **Purpose:** A modular, low-latency pipeline for handling raw audio byte streams.
- **Key Modules:**
  - `VAD/vad_handler.py`: Voice Activity Detection (silence detection).
  - `STT/faster_whisper_handler.py`: Local speech transcription.
  - `pipeline/s2s_pipeline.py`: Orchestrates threaded event loops.

6. Configuration & Environment Variables

Variable / Secret	Type	Default Value	Description
`GEMINI_API_KEY`	`string`	*(Empty)*	Required Google API key to authenticate with `gemini-flash-latest`. Loaded via `os.environ`.
`PORT`	`integer`	`8000`	Port for the Uvicorn ASGI server.
`HOST`	`string`	`0.0.0.0`	Host binding for Uvicorn.
`CHROMA_PERSIST_DIR`	`path`	`./backend/chroma_db`	Path where Chroma SQLite files are stored.

Note: For security, `GEMINI_API_KEY` must never be hardcoded. It is currently injected at runtime via `run_local.bat` or `.env` files.

---

7. Reliability, Security & Error Handling

Security & Guardrails

- **Prompt Injection Defense:** The `LibraryRAG` engine utilizes a strict system prompt instructing the agent to *only* answer based on retrieved context. Out-of-bounds queries gracefully fallback to a canned "I cannot help with that" response.
- **Secrets Management:** The repository adheres strictly to GitHub Push Protection policies. No API keys are tracked in version control; they are strictly relegated to local `run_local.bat` injection.
- **Path Traversal Protection:** The FastAPI backend endpoints do not accept raw file paths, preventing arbitrary local file reads.

Reliability & Error Handling

- **Missing Credentials:** If `GEMINI_API_KEY` is missing or invalid, the backend gracefully catches the `401 Unauthorized` or `404 Not Found` API core exceptions, preventing a hard server crash, and returns a 500 status to the client frontend with a standardized "Cannot reach backend" generic message to obfuscate backend errors from standard users.

- **LLM Rate Limits (HTTP 429):** Integrations natively utilize LangChain's retry semantics and Tenacity retry blocks to exponentially backoff when Google API rate limits are hit.
- **Database Idempotency:** The RAG initialization skips expensive document embedding and generation if the `chroma_db` directory is detected, reducing startup time from minutes to milliseconds.